CS-375 Operating Systems Lab 8: Caching, TLB, and Multi-level Page Tables

Santiago Soto

October 31, 2026

Contents

1 Design Overview **2**
1.1 2-Level Page Table . . . 2
1.2 TLB Structure and LRU Logic . . . 2
1.3 Flowchart: VA to PA . . . 3

2 Implementation Highlights **3**
2.1 How Page Faults Are Simulated . . . 3
2.2 LRU in the TLB (Timestamp Method) . . . 4
2.3 Changes from the Provided Skeleton . . . 4

3 Screenshots **5**
3.1 Build . . . 5
3.2 Part 1: Direct-Mapped Page Table . . . 6
3.3 Part 2: 2-Level Page Table . . . 6
3.4 Part 3: TLB . . . 7
3.5 Terminal Output: First 10 and Last 10 Lines . . . 7
3.6 PAGE FAULT Messages . . . 8
3.7 TLB Hit Rate . . . 9
3.8 Without the TLB . . . 9

4 Performance Analysis **10**
4.1 Summary Metrics . . . 10
4.2 Translation Time With and Without the TLB . . . 10
4.3 TLB Hit Rate vs. Working Set Size . . . 11

5 Experiment: TLB Size Impact **12**

6 Questions **14**

7 Conclusion **15**

1 Design Overview

The simulator in `main.cpp` translates 32-bit virtual addresses with 4 KB pages. Each part of the lab is a mode of the same program, chosen with `--part`, and Part 4 is the default.

Part	Command	What runs
1	`./lab --part 1`	Direct-mapped table with 2^{20} entries.
2	`./lab --part 2`	2-level table, created on demand.
3	`./lab --part 3`	2-level table with a 16-entry LRU TLB.
4	`./lab`	Part 3 limited to 256 frames, with random invalidation and eviction.

Table 1: Modes of the simulator.

1.1 2-Level Page Table

A virtual address splits into a 10-bit directory index, a 10-bit table index and a 12-bit offset. The directory has 1024 entries, each pointing to a second-level table of 1024 entries, which gives $1024 \times 1024 = 2^{20}$ virtual pages. Second-level tables are allocated only when a region is first used.

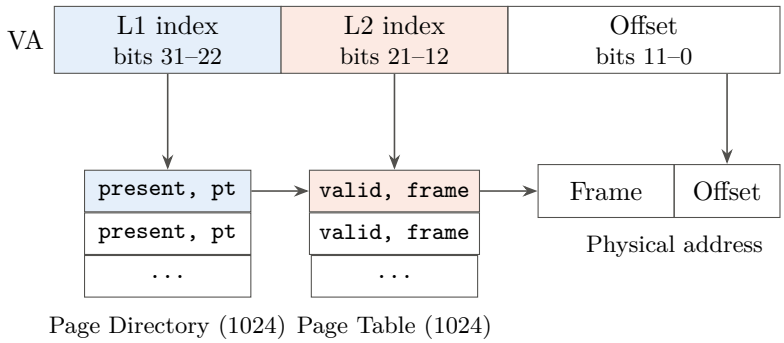


Figure 1: The two lookups of a page table walk. The L1 index selects a directory entry, which points to a page table. The L2 index selects the page table entry, which holds the frame. The offset is copied unchanged.

1.2 TLB Structure and LRU Logic

The TLB is fully associative with 16 entries. Each entry stores `vpn`, `pfn`, `timestamp` and `valid`, and any page can sit in any entry. A lookup compares the VPN with every valid entry. The TLB keeps a clock that increases on every lookup hit and every insert, and that access stamps the entry with the new clock value. The entry with the smallest timestamp is therefore the least recently used, and it is replaced on insert. An empty slot has timestamp 0, so it is always filled before a live entry is replaced.

1.3 Flowchart: VA to PA

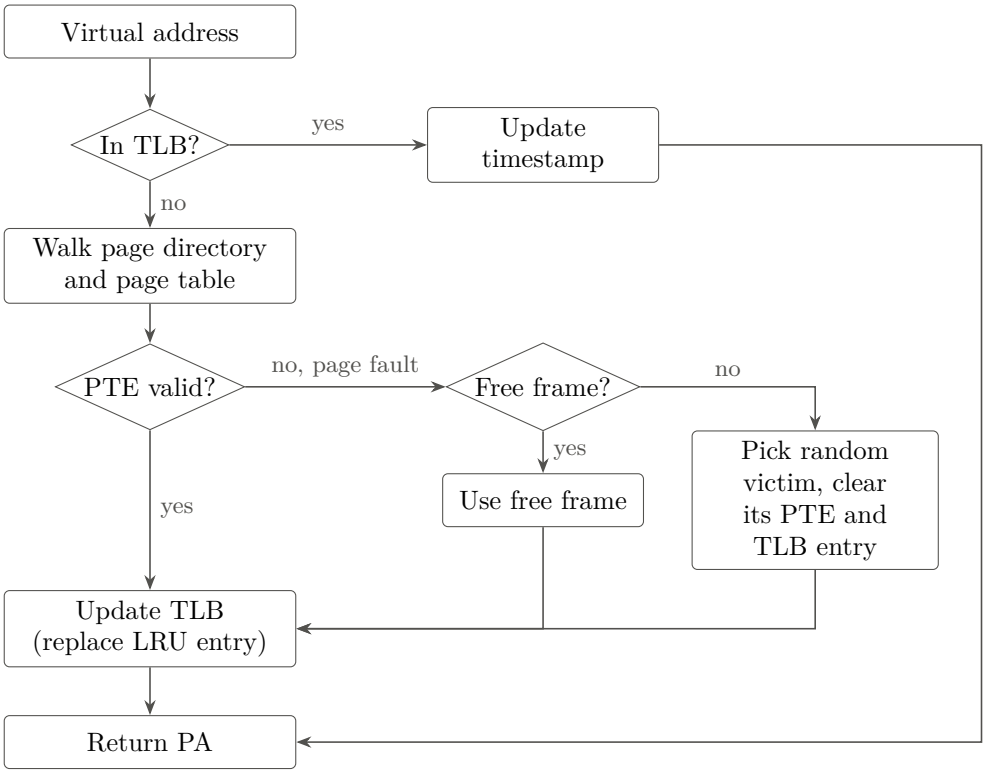


Figure 2: Translation flow in Part 4.

2 Implementation Highlights

2.1 How Page Faults Are Simulated

Part 4 allows only 256 frames. Three lists track physical memory: `free_frames`, `allocated_frames`, and `frame_owner`, which maps each frame back to the VPN stored in it.

A fault happens when a walk finds an invalid page table entry, either on the first access to a page or after the page lost its frame. Pages lose their frames in two ways:

- **Random invalidation.** Before each access there is a 1% chance that a random resident page is invalidated and its frame is freed. This stands in for the operating system reclaiming a page.
- **Eviction.** If a fault finds no free frame, a random allocated frame is chosen as the victim and its current page is removed.

Removing a page must clear both the page table entry and any TLB entry for that page. Otherwise the TLB would keep sending the old page to a frame that now holds different data.

```
uint32_t allocate_frame(uint32_t vpn, bool& evicted) {
    uint32_t frame;
    evicted = free_frames.empty();
    if (evicted) {
        frame = allocated_frames[random_allocated_slot()];
```

```
        unmap_frame(frame);    // clears the PTE and the TLB entry
        stats.evictions++;
    } else {
        frame = free_frames.back();
        free_frames.pop_back();
        allocated_frames.push_back(frame);
    }
    frame_owner[frame] = vpn;
    return frame;
}
```

2.2 LRU in the TLB (Timestamp Method)

A hit refreshes the entry's timestamp, and an insert scans for the smallest one. Both operations take $O(n)$ time in the number of entries, which is acceptable for 16 entries.

```
void insert(uint32_t vpn, uint32_t pfn) {
    size_t lru_idx = 0;
    uint64_t oldest = entries[0].timestamp;
    for (size_t i = 1; i < entries.size(); i++) {
        if (entries[i].timestamp < oldest) {
            oldest = entries[i].timestamp;
            lru_idx = i;
        }
    }
    entries[lru_idx].vpn = vpn;
    entries[lru_idx].pfn = pfn;
    entries[lru_idx].timestamp = ++clock;
    entries[lru_idx].valid = true;
}
```

The TLB holds its entries in a vector sized by a constructor argument instead of a fixed `entries[16]` array. The default stays 16, and Section 5 uses the same class for 4, 8 and 32 entries.

2.3 Changes from the Provided Skeleton

The skeleton needed these corrections to complete the lab.

Problem in the skeleton	Change
`allocate_frame` loops forever once all 256 frames are used, and this trace touches 541 pages.	Added eviction with a random victim.
Eviction did not exist, so nothing removed a page from the TLB.	Added `TLB::invalidate`, called whenever a page loses its frame.
`fin >> hex >> addr` fails on `0x7fff5fbff000`, which is 48 bits wide, and silently ends the input.	Each address is read as 64 bits. Wider addresses print a warning and use their low 32 bits.
`hex` stays set on `cout`, so summary counts print in hexadecimal.	Hexadecimal formatting is confined to a helper, `hex32`.
`make run` runs `./generate` without building it.	`addresses.txt` is now a Makefile target that depends on `generate`.
Timing the printing loop measures terminal output rather than translation.	Time is measured separately with printing off (Section 4).

Table 2: Corrections to the skeleton.

Two details of the handout are also worth noting. Its example address `0x7fff5fbff000` does not fit in a 32-bit address space, so the simulator uses `0x5fbff000` and page `0x5fbff`. The comment in `generate_addresses.cpp` called the hot region 100 pages, but the range `0x70000000` to `0x700FFFFF` is 1 MB, which is 256 pages. The code is unchanged and the comment is corrected.

3 Screenshots

3.1 Build

```
root@Saint-James:~/Lab-8-Caching-TLB-and-Multi-level-Page-Tables# cd ~/Lab-8-Caching-TLB-and-Multi-level-Page-Tables/src
make clean && make
rm -f lab generate addresses.txt
g++ -std=c++17 -O2 -Wall -o lab main.cpp
g++ -std=c++17 -O2 -Wall -o generate generate_addresses.cpp
./generate
root@Saint-James:~/Lab-8-Caching-TLB-and-Multi-level-Page-Tables/src#
```

Figure 3: Building the lab with the provided compiler flags.

3.2 Part 1: Direct-Mapped Page Table

```
root@Saint-James:~/Lab-8-Caching-TLB-and-Multi-level-Page-Tables/src# cd ~/Lab-8-Caching-TLB-and-Multi-leve
l-Page-Tables/src
./lab --part 1 --file sample_addresses.txt
WARNING: 0x7fff5fbff000 is wider than 32 bits, using 0x5fbff000
Starting translation with direct-mapped page table

VA: 0x00000000 → PA: 0x00000000 (page 0)
VA: 0x5fbff000 → PA: 0x00001000 (page 0x5fbff)
VA: 0x00001abc → PA: 0x00002abc (page 0x1)
VA: 0x00001def → PA: 0x00002def (page 0x1)
VA: 0x70000123 → PA: 0x00003123 (page 0x70000)
VA: 0xffffffff → PA: 0x00004fff (page 0xfffff)

=== SUMMARY ===
Total Addresses: 6
Page Faults: 5 (evictions: 0, invalidations: 0)
TLB: disabled
Avg Translation Cost: 100.00 modeled cycles
Avg Translation Time: 0.0164 µs (measured with printing off)
Page Table Memory: 8192 KB (always)
root@Saint-James:~/Lab-8-Caching-TLB-and-Multi-level-Page-Tables/src#
```

Figure 4: Part 1 on the handout's sample addresses, including the warning for the 48-bit address.

The first address maps to physical address 0 as in the handout, because frames are assigned in order on first use. Six addresses cause only 5 faults, since `0x00001abc` and `0x00001def` are in the same page: both use the same frame and keep their own offsets. Every translation costs 100 modeled cycles, because a flat table needs one memory access.

3.3 Part 2: 2-Level Page Table

```
root@Saint-James:~/Lab-8-Caching-TLB-and-Multi-level-Page-Tables/src# cd ~/Lab-8-Caching-TLB-and-Multi-leve
l-Page-Tables/src
./lab --part 2 | tail -n 9
VA: 0x7407979a → PA: 0x0021c79a (page 0x74079)

=== SUMMARY ===
Total Addresses: 1000
Page Faults: 541 (evictions: 0, invalidations: 0)
TLB: disabled
Avg Translation Cost: 200.00 modeled cycles
Avg Translation Time: 0.0158 µs (measured with printing off)
Page Table Memory: 2072 KB in 257 tables (flat table: 8192 KB)
root@Saint-James:~/Lab-8-Caching-TLB-and-Multi-level-Page-Tables/src#
```

Figure 5: Part 2 summary, with page table memory compared to the flat table.

Part 2 gives the same translations as Part 1 for the same addresses, but the two designs trade time against space. The 2-level table costs 200 cycles per translation, twice the flat table, because it needs two memory accesses. In exchange it uses 2072 KB instead of 8192 KB, because only 257 of the 1024 second-level tables were created. The saving is modest here because random addresses touch many regions. A program with a compact address space would need only a few tables.

3.4 Part 3: TLB

```
root@Saint-James:~/Lab-8-Caching-TLB-and-Multi-level-Page-Tables/src# cd ~/Lab-8-Caching-TLB-and-Multi-leve
l-Page-Tables/src
./lab --part 3 | grep "TLB HIT" | head -n 5
./lab --part 3 --quiet
VA: 0x70027ef4 → PA: 0x00008ef4 [TLB HIT]
VA: 0x70099e2a → PA: 0x00006e2a [TLB HIT]
VA: 0x7000e70f → PA: 0x0000b70f [TLB HIT]
VA: 0x7002ef39 → PA: 0x00003f39 [TLB HIT]
VA: 0x70005e76 → PA: 0x00012e76 [TLB HIT]
Starting translation with 2-level page table + TLB (16-entry LRU)


=== SUMMARY ===
Total Addresses: 1000
Page Faults: 541 (evictions: 0, invalidations: 0)
TLB Hits: 43, Misses: 957, Hit Rate: 4.30%
Avg Translation Cost: 192.40 modeled cycles
Avg Translation Time: 0.0418 µs (measured with printing off)
Page Table Memory: 2072 KB in 257 tables (flat table: 8192 KB)
root@Saint-James:~/Lab-8-Caching-TLB-and-Multi-level-Page-Tables/src#
```

Figure 6: Part 3 TLB hits and summary.

Only 43 of 1000 translations hit. All of them fall in the first 700 addresses, where $43/700 = 6.1\%$ closely matches the 6.25% expected from 16 entries covering 256 hot pages. The last 300 addresses never hit, because each goes to a different page. The average cost matches the formula in Section 4: $1 + 0.957 \times 200 = 192.40$ cycles.

3.5 Terminal Output: First 10 and Last 10 Lines

```
root@Saint-James:~/Lab-8-Caching-TLB-and-Multi-level-Page-Tables/src# cd ~/Lab-8-Caching-TLB-and-Multi-leve
l-Page-Tables/src
./lab | head -n 10
Starting translation with 2-level page table + TLB (16-entry LRU) + 256 frames

PAGE FAULT: VA 0x7005fe1d → free frame 214
VA: 0x7005fe1d → PA: 0x000d6e1d [TLB MISS]
PAGE FAULT: VA 0x700cbea3 → free frame 128
VA: 0x700cbea3 → PA: 0x00080ea3 [TLB MISS]
PAGE FAULT: VA 0x700f3620 → free frame 235
VA: 0x700f3620 → PA: 0x000eb620 [TLB MISS]
PAGE FAULT: VA 0x7002ef59 → free frame 219
VA: 0x7002ef59 → PA: 0x000dbf59 [TLB MISS]
root@Saint-James:~/Lab-8-Caching-TLB-and-Multi-level-Page-Tables/src#
```

Figure 7: First 10 lines of the Part 4 output.

The run starts with empty memory, so the first accesses are all page faults that take a free frame, followed by a TLB miss. Frame numbers look random because Part 4 shuffles the free list.

```
root@Saint-James:~/Lab-8-Caching-TLB-and-Multi-level-Page-Tables/src# cd ~/Lab-8-Caching-TLB-and-Multi-leve
l-Page-Tables/src
./lab | tail -n 10
PAGE FAULT: VA 0x7407979a → evicted frame 7
VA: 0x7407979a → PA: 0x0000779a [TLB MISS]

=== SUMMARY ===
Total Addresses: 1000
Page Faults: 544 (evictions: 281, invalidations: 7)
TLB Hits: 43, Misses: 957, Hit Rate: 4.30%
Avg Translation Cost: 192.40 modeled cycles
Avg Translation Time: 0.0748 µs (measured with printing off)
Page Table Memory: 2072 KB in 257 tables (flat table: 8192 KB)
root@Saint-James:~/Lab-8-Caching-TLB-and-Multi-level-Page-Tables/src#
```

Figure 8: Last 10 lines of the Part 4 output, with the summary.

By the end of the run memory is full, so faults evict a frame instead of taking a free one.

3.6 PAGE FAULT Messages

```
root@Saint-James:~/Lab-8-Caching-TLB-and-Multi-level-Page-Tables/src# cd ~/Lab-8-Caching-TLB-and-Multi-leve
l-Page-Tables/src
./lab | grep "evicted" | head -n 10
./lab | grep -c "PAGE FAULT"
PAGE FAULT: VA 0xa2eb589f → evicted frame 24
PAGE FAULT: VA 0xf0b2edda → evicted frame 110
PAGE FAULT: VA 0xcd0b0353 → evicted frame 23
PAGE FAULT: VA 0x93524697 → evicted frame 189
PAGE FAULT: VA 0xad5ae78b → evicted frame 5
PAGE FAULT: VA 0x635f1ab1 → evicted frame 202
PAGE FAULT: VA 0x92c82ea2 → evicted frame 161
PAGE FAULT: VA 0xa4ae897b → evicted frame 123
PAGE FAULT: VA 0x20e56620 → evicted frame 43
PAGE FAULT: VA 0x75500fa0 → evicted frame 77
544
root@Saint-James:~/Lab-8-Caching-TLB-and-Multi-level-Page-Tables/src#
```

Figure 9: Page faults that evicted a frame, and the total number of faults.

The fault counts add up. The trace touches 541 distinct pages, so that many faults were unavoidable, and the other few came from pages that were invalidated or evicted and then used again. Of the 544 faults, 263 found a free frame: the 256 frames available at the start, plus the 7 freed by random invalidation. The remaining 281 had to evict.

3.7 TLB Hit Rate

```
root@Saint-James:~/Lab-8-Caching-TLB-and-Multi-level-Page-Tables/src# cd ~/Lab-8-Caching-TLB-and-Multi-leve
l-Page-Tables/src
./lab --runs 10
Run  Seed  Hit Rate  Page Faults
  1    42     4.30%          544
  2    43     4.30%          543
  3    44     4.40%          548
  4    45     4.30%          542
  5    46     4.30%          548
  6    47     4.30%          547
  7    48     4.30%          548
  8    49     4.30%          543
  9    50     4.30%          546
 10    51     4.30%          546

Average over 10 runs: Hit Rate 4.31% (min 4.30%, max 4.40%), Page Faults 545.5
root@Saint-James:~/Lab-8-Caching-TLB-and-Multi-level-Page-Tables/src#
```

Figure 10: TLB hit rate averaged over 10 random seeds.

The hit rate stays between 4.30% and 4.40%, averaging 4.31%. The seed changes only which pages are invalidated and evicted, and the address trace stays fixed. It also matches Part 3, which has no memory limit. The first eviction happens at about address 716, after the last hit, so the frame limit never removes a page that was about to hit.

3.8 Without the TLB

```
root@Saint-James:~/Lab-8-Caching-TLB-and-Multi-level-Page-Tables/src# cd ~/Lab-8-Caching-TLB-and-Multi-leve
l-Page-Tables/src
./lab --no-tlb --quiet
Starting translation with 2-level page table + 256 frames


=== SUMMARY ===
Total Addresses: 1000
Page Faults: 544 (evictions: 281, invalidations: 7)
TLB: disabled
Avg Translation Cost: 200.00 modeled cycles
Avg Translation Time: 0.0361 µs (measured with printing off)
Page Table Memory: 2072 KB in 257 tables (flat table: 8192 KB)
root@Saint-James:~/Lab-8-Caching-TLB-and-Multi-level-Page-Tables/src#
```

Figure 11: The same run with the TLB turned off.

The page fault count is unchanged, because a TLB only caches translations and has no effect on which pages are in memory. The cost is exactly 200 cycles, since every translation now walks the table.

4 Performance Analysis

4.1 Summary Metrics

These are the results for `addresses.txt` in the default Part 4 configuration with seed 42. Averages use seeds 42 to 51.

Metric	Value
TLB Hit Rate	4.30%
TLB Hit Rate (average of 10 runs)	4.31%
Page Faults	544
Page Faults (average of 10 runs)	545.5
Evictions	281
Random Invalidations	7
Avg Translation Time	0.0592 μs
Avg Translation Cost	192.40 modeled cycles

Table 3: Summary metrics for `addresses.txt`.

Page faults. The 544 faults are high for 1000 addresses because the trace is dominated by first accesses. It touches 541 distinct pages, and a page must fault the first time it is used, no matter how much memory is available. More memory would remove the evictions but not those faults.

TLB hit rate. The 4.30% hit rate is low because the trace gives the TLB little to work with. Hot addresses are spread evenly over 256 pages, so a 16-entry TLB holds about $16/256 = 6.25\%$ of them, and the other 300 addresses are scattered over the whole address space. Every address also picks a random page, so a program's usual habit of reusing the current page is missing. This is a property of the trace rather than a fault in the TLB.

To show the TLB under realistic conditions, the experiments also use a 20,000-address locality trace in which 90% of accesses step through a 64-page array in 64-byte strides and 10% jump to random pages.

4.2 Translation Time With and Without the TLB

Cycles use a simple cost model: a TLB lookup takes 1 cycle and a memory access takes 100 cycles. A 2-level walk needs 2 memory accesses. Without a TLB every translation costs 200 cycles. With a TLB of hit rate h, the average is

$$C = 1 + (1 - h) \times 200 \text{ cycles.}$$

Page fault service time is left out, since it is the same in both cases.

	Modeled cycles		Measured μs	
Trace	**With TLB**	**Without**	**With TLB**	**Without**
`addresses.txt`	192.38	200.00	0.0592	0.0301
Locality trace	22.71	200.00	0.0304	0.0174

Table 4: Translation cost per address with a 16-entry TLB. Measured times change from run to run, so they differ from the values shown in the screenshots.

The modeled cycles are the meaningful comparison. With locality, the TLB reduces translation cost from 200.00 to 22.71 cycles, almost a factor of nine. On `addresses.txt` the saving is small, as the hit rate predicts. The two results confirm that a TLB's value depends on the program's locality.

The measured times show the opposite order, and that does not contradict the model. They measure this C++ simulator running on a modern CPU, not a memory system. Here a page table walk is just two array reads, which the host's own caches serve quickly, while the simulated TLB is a loop over 16 entries in software. A hardware TLB compares all its entries in parallel, and a real walk costs real memory accesses, so the modeled cycles describe the hardware case. The timings were taken with printing off and after an untimed warm-up pass, so they do not include terminal output or page table allocation.

4.3 TLB Hit Rate vs. Working Set Size

This experiment accesses a chosen number of pages uniformly at random, 50,000 times, with a 16-entry TLB, 256 frames and no random invalidation.

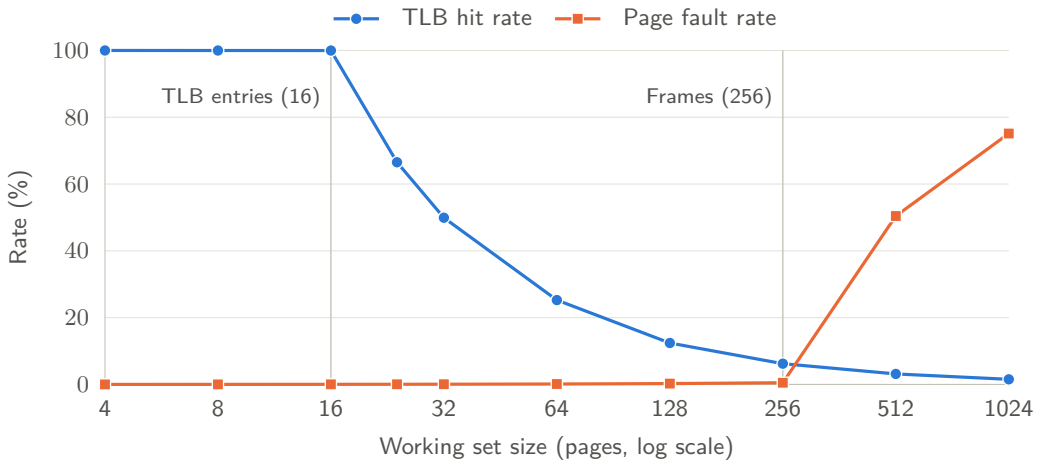


Figure 12: TLB hit rate and page fault rate against working set size.

Pages	TLB Hit Rate (%)	Page Fault Rate (%)
4	99.99	0.01
8	99.98	0.02
16	99.97	0.03
24	66.52	0.05
32	49.91	0.06
64	25.26	0.13
128	12.40	0.26
256	6.21	0.51
512	3.15	50.43
1,024	1.53	75.12

Table 5: Working set experiment, average of 10 runs.

The graph has two clear limits. While the working set fits in the 16 TLB entries, the hit rate is almost 100%. Beyond that it follows $16/w$ for a working set of w pages, because LRU keeps 16 of the w equally likely pages. The second limit is physical memory. Up to 256 pages, faults are first accesses only, at most 0.51%. Past 256 pages the fault rate jumps to 50.43% at 512 pages and keeps rising, which Question 2 discusses.

The measured values match these predictions closely. At 24 pages the hit rate is 66.52% against a predicted $16/24 = 66.67\%$, and at 512 pages the fault rate is 50.43% against a predicted 50%. Uniform access is the worst case for the TLB, since no page is more likely than another. It shows the TLB's reach clearly: 16 entries of 4 KB each cover only 64 KB of memory.

```
root@Saint-James:~/Lab-8-Caching-TLB-and-Multi-level-Page-Tables/src# cd ~/Lab-8-Caching-TLB-and-Multi-leve
l-Page-Tables/src
./lab --experiment | tail -n +10
=== Experiment: Working Set Size (16-entry TLB, 256 frames, no invalidation) ===
Pages   TLB Hit Rate   Page Fault Rate
    4         99.99%             0.01%
    8         99.98%             0.02%
   16         99.97%             0.03%
   24         66.52%             0.05%
   32         49.91%             0.06%
   64         25.26%             0.13%
  128         12.40%             0.26%
  256          6.21%             0.51%
  512          3.15%            50.43%
 1024          1.53%            75.12%

=== Measured Simulator Time (printing off) ===
                With TLB    Without TLB
addresses.txt    0.0569 µs     0.0256 µs
locality trace   0.0228 µs     0.0158 µs
root@Saint-James:~/Lab-8-Caching-TLB-and-Multi-level-Page-Tables/src#
```

Figure 13: Working set experiment and measured simulator time.

5 Experiment: TLB Size Impact

The TLB was run with 4, 8, 16 and 32 entries in the Part 4 configuration, using the same 10 seeds for each size.

	`addresses.txt`		Locality trace	
Entries	**Hit Rate (%)**	**Cycles**	**Hit Rate (%)**	**Cycles**
4	0.70	199.60	88.75	23.50
8	2.01	196.98	88.86	23.27
16	4.31	192.38	89.14	22.71
32	8.40	184.20	89.71	21.58

Table 6: TLB size results, average of 10 runs. Without a TLB, both traces cost 200.00 cycles per translation.

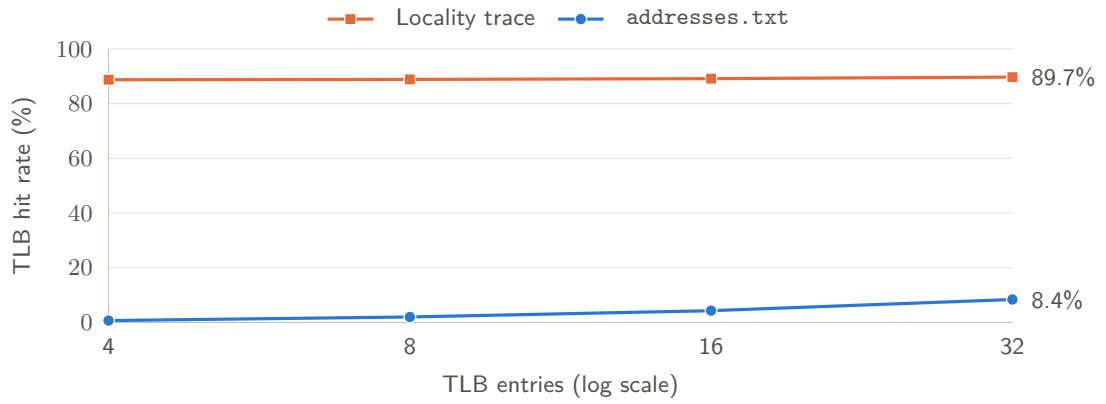


Figure 14: TLB hit rate against TLB size.

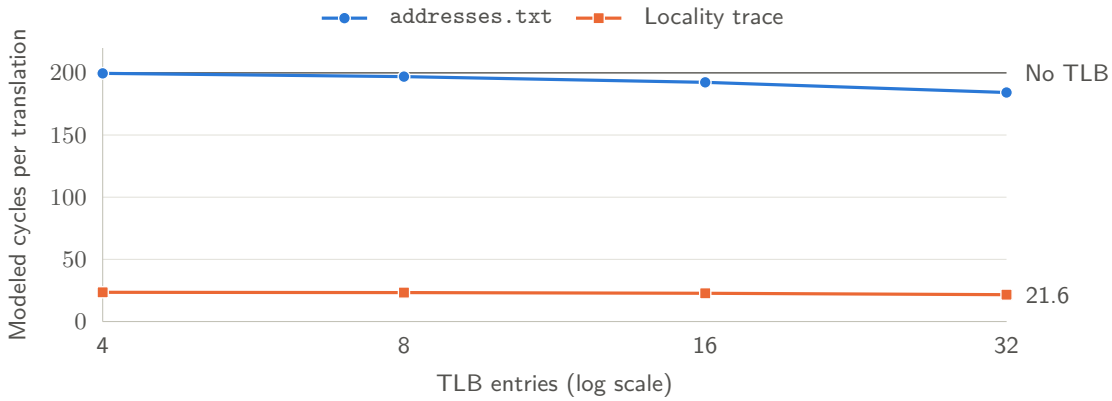


Figure 15: Modeled translation cost against TLB size. The line at 200 cycles is the cost without a TLB.

The two traces respond differently. On `addresses.txt` the hit rate roughly doubles each time the TLB doubles, which is the $w > 16$ behavior from Figure 12: with no locality, the hit rate is proportional to the fraction of pages the TLB can hold. On the locality trace the hit rate barely moves, staying near the 88.6% that spatial locality alone provides ($0.9 \times 63/64$), since the sequential scan makes 63 of every 64 accesses to the page it just used. Even a 4-entry TLB captures that.

The cycle results follow directly from the hit rates through the cost formula. At 16 entries, $1 + (1 - 0.0431) \times 200 = 192.38$ cycles for `addresses.txt`, and $1 + (1 - 0.8914) \times 200 = 22.72$ cycles for the locality trace, which is within rounding of the measured 22.71. Going from 4 to 32 entries saves 15.40 cycles per translation on `addresses.txt` but only 1.92 on the locality trace. A larger TLB helps when the program's active pages exceed the TLB, and adds little once they already fit. Since a larger TLB costs more hardware, the locality result explains why real TLBs can stay small.

```
root@Saint-James:~/Lab-8-Caching-TLB-and-Multi-level-Page-Tables/src# cd ~/Lab-8-Caching-TLB-and-Multi-leve
l-Page-Tables/src
./lab --experiment | head -n 8
=== Experiment: TLB Size Impact (average of 10 runs) ===
               addresses.txt         locality trace
TLB Entries   Hit Rate   Cycles     Hit Rate   Cycles
       none          -   200.00            -   200.00
          4      0.70%   199.60       88.75%    23.50
          8      2.01%   196.98       88.86%    23.27
         16      4.31%   192.38       89.14%    22.71
         32      8.40%   184.20       89.71%    21.58
root@Saint-James:~/Lab-8-Caching-TLB-and-Multi-level-Page-Tables/src#
```

Figure 16: TLB size experiment from the terminal.

6 Questions

Question 1

Why does TLB improve performance?

Every memory reference needs a translation, and a walk of a 2-level table takes two extra memory accesses before the program's own access can begin. The TLB stores recent translations in a small, fast cache, so a hit avoids the walk. Programs tend to reuse the same pages (temporal locality) and access nearby addresses that share a page (spatial locality), so a few entries cover most accesses.

In the cost model, a hit costs 1 cycle and a walk costs 200. Solving $1 + (1 - h) \times 200 < 200$ shows the TLB pays for itself at a hit rate of only 0.5%. In the experiments, the locality trace falls from 200.00 to 22.71 cycles per translation, while `addresses.txt`, which has no real locality, falls only to 192.38 cycles. The benefit comes from locality in the program as much as from the TLB.

Question 2

What happens if working set > physical memory?

The system starts thrashing. Pages the program still needs are evicted to make room for others, soon fault back in, and evict something else in turn. Time goes to moving pages instead of running the program, and because each fault is far slower than a memory access, performance collapses.

Figure 12 shows this. With 256 frames, a working set of 256 pages has a fault rate of 0.51%, all of it first accesses. Doubling the working set to 512 pages raises it to 50.43%, and 1024 pages raises it to 75.12%. With uniform access this matches $1 - 256/w$, the chance that the requested page is not resident. The TLB gives no protection, because it can only speed up translations for pages that are in memory. Remedies are to add memory, reduce the number of running processes so each one's working set fits, or use a better replacement policy, though no policy helps much when a program truly needs more pages than exist.

Question 3

How would you implement LRU with linked list + hash?

Use a doubly linked list ordered by recency, with the most recently used entry at the head and the least recently used at the tail, plus a hash map from VPN to that entry's list node.

- **Lookup:** find the VPN in the hash map in $O(1)$. On a hit, unlink the node and move it to the head. This is also $O(1)$, since a doubly linked node can be removed without searching.
- **Insert:** if the TLB is full, remove the tail node and erase its VPN from the map. Then add the new node at the head and add it to the map.
- **Invalidate:** find the node through the map, unlink it and erase it.

```
struct Node { uint32_t vpn, pfn; };
list<Node> order;                                   // front = most recent
unordered_map<uint32_t, list<Node>::iterator> map;

bool lookup(uint32_t vpn, uint32_t& pfn) {
    auto it = map.find(vpn);
    if (it == map.end()) return false;
    order.splice(order.begin(), order, it->second);  // move to front
    pfn = it->second->pfn;
    return true;
}

void insert(uint32_t vpn, uint32_t pfn) {
    if (order.size() == capacity) {
        map.erase(order.back().vpn);
        order.pop_back();
    }
    order.push_front({vpn, pfn});
    map[vpn] = order.begin();
}
```

Every operation is $O(1)$, compared with the $O(n)$ scan of the timestamp method. The structure uses more memory per entry, and for 16 entries the simple scan is likely just as fast in software. The linked list and hash design pays off at larger sizes, such as an operating system's page replacement list. Hardware TLBs use neither approach: they compare entries in parallel and usually approximate LRU with a few bits per entry.

7 Conclusion

The results show four main points.

- **The 2-level table trades time for space.** It doubles the memory accesses per walk but reduced page table memory from 8192 KB to 2072 KB.
- **The TLB removes that time cost when the program has locality.** Cost fell from 200.00 to 22.71 cycles on the locality trace, but only to 192.38 on `addresses.txt`.
- **TLB size matters less than locality.** Going from 4 to 32 entries barely changed the locality trace's hit rate.
- **Physical memory sets a hard limit.** Once the working set passed 256 pages, the fault rate rose from 0.51% to 50.43%.